

BİLGİSAYAR AĞLARI PROJESİ · 2026

gRPC Mikroservisler

Node.js, Python ve Envoy Proxy ile inşa edilmiş
yüksek performanslı çok dilli mikroservis mimarisi

gRPC · PROTOCOL BUFFERS

NODE.JS · PYTHON

ENVOY PROXY · DOCKER

emrebykdr

github.com/emrebykdr/gRPC-Microservices

⚡ Problem

Modern uygulamalar onlarca bağımsız servis içerir.

Bu servisler nasıl **hızlı, güvenli ve verimli** iletişim kurar?

- **REST/HTTP+JSON** her istekte metin ayrıştırır — yavaş ve şemasız

- Farklı diller (Node.js ↔ Python) arasında **tip güvenliği yoktur**

- Tarayıcılar **HTTP/2'yi** gRPC ile doğrudan kullanamaz

✦ Çözüm

gRPC + Protocol Buffers ile binary iletişim, güçlü tip sistemi ve streaming desteği

- **.proto** dosyası tek kaynak — tüm diller aynı sözleşmeyle çalışır

- Binary serialization: JSON'dan **7–10× daha hızlı** ve daha küçük

- **Envoy Proxy** tarayıcı trafiğini native gRPC'ye dönüştürür

Neden gRPC seçtik?

Google tarafından geliştirilen, HTTP/2 tabanlı modern RPC framework'ü



Performans

Protobuf binary format ile JSON'dan çok daha hızlı serileştirme ve küçük paket boyutu



Tip Güvenliği

.proto şeması kodları otomatik üretir — runtime hataları derleme zamanında yakalanır



Çok Dilli

Aynı .proto'dan Node.js, Python, Go, Java stubs otomatik üretilir — polyglot mimari



Streaming

4 farklı RPC tipi: Unary, Server Streaming, Client Streaming, Bidirectional

REST + JSON

~2,400 req/s



gRPC + Protobuf

~14,000 req/s

~5.8× daha hızlı

HTTP/2 neden kritik?

gRPC, HTTP/1.1'in sınırlarını aşmak için HTTP/2'yi zorunlu kılıyor. İşte farkı yaratan özellikler:



Multiplexing

Tek TCP bağlantısı üzerinden birden fazla istek/yanıt **paralel** akar. HTTP/1.1'deki "head-of-line blocking" sorunu ortadan kalkar.



Header Compression (HPACK)

HTTP başlıkları sıkıştırılarak gönderilir. Tekrarlayan başlıklar sadece **delta olarak** iletilir — bandwidth %60-80 azalır.



Binary Framing

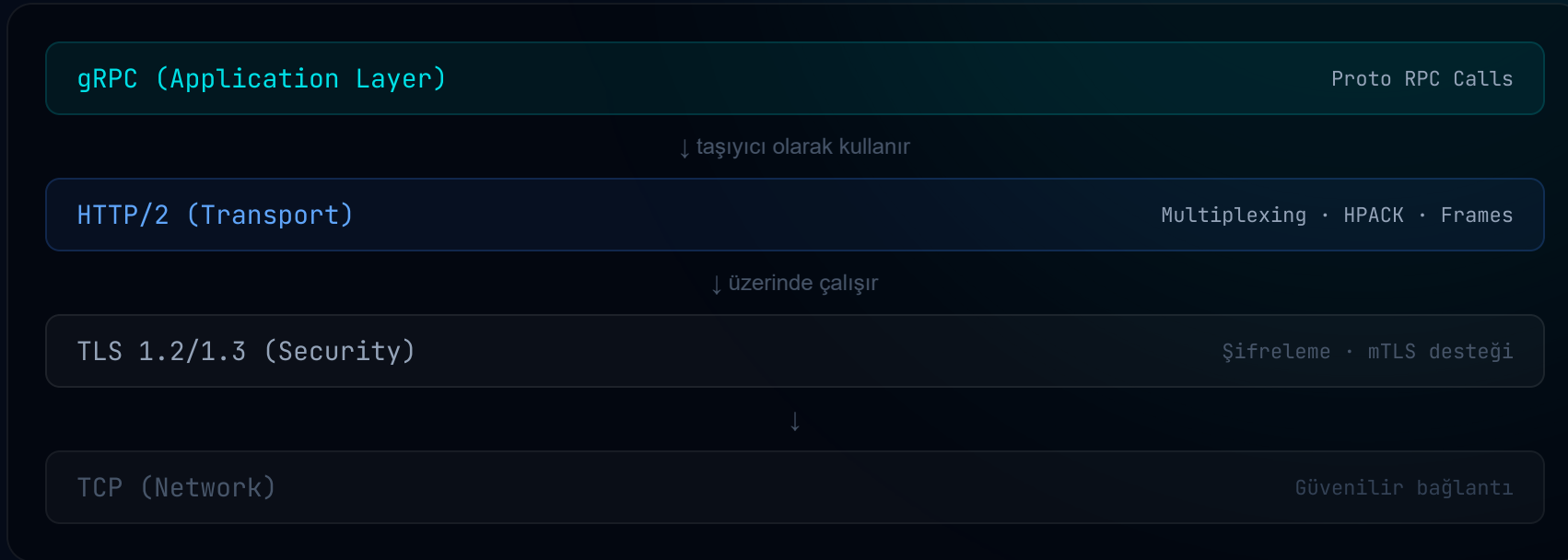
HTTP/1.1 metin tabanlıdır; HTTP/2 binary frame'ler kullanır. Protobuf binary verisi ile mükemmel uyum — **ayırıştırma overhead'i** sıfır.



Bidirectional Streaming

İstemci ve sunucu **eş zamanlı** veri gönderebilir. Server Push sayesinde istemci istemeden veri akışı başlar.

PROTOKOL KATMANLARI



ÖZELLİK	REST/HTTP1.1	GRPC/HTTP2
Protokol	Metin	Binary
Multiplexing	Yok	Var
Header boyutu	~800 byte	~50 byte
Streaming	Kısıtlı	4 Mod
Tip güvenliği	Yok	Tam
Code-gen	Manuel	Otomatik

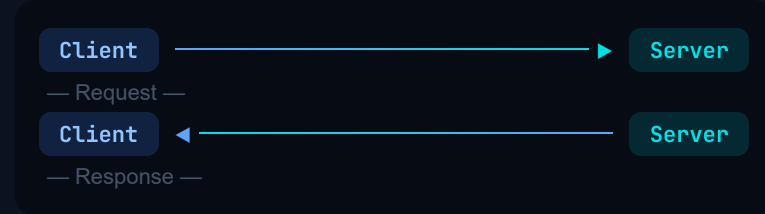
gRPC'nin 4 İletişim Modeli

Her model farklı bir ihtiyaca cevap verir — projemizde hepsi tanımlandı

Unary

Tekil İstek-Yanıt

Klasik HTTP gibi: 1 istek → 1 yanıt



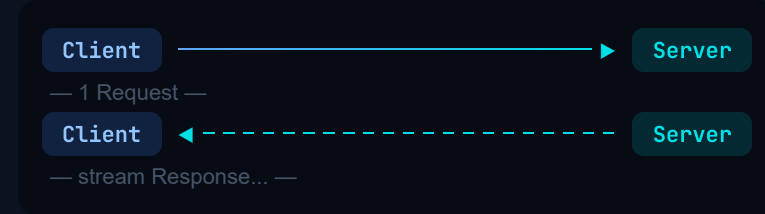
rpc `CreateOrder`(Req) returns(Res)

Projemizde: **Sipariş oluşturma**, stok kontrolü, bildirim gönderme

Server Streaming

Sunucu Akışı

1 istek → sürekli yanıt akışı



rpc `TrackOrder`(Req) returns(stream Res)

Projemizde: **Sipariş takibi** — RECEIVED → PROCESSING → DELIVERED

Client Streaming

İstemci Akışı

Çoklu istek → 1 yanıt



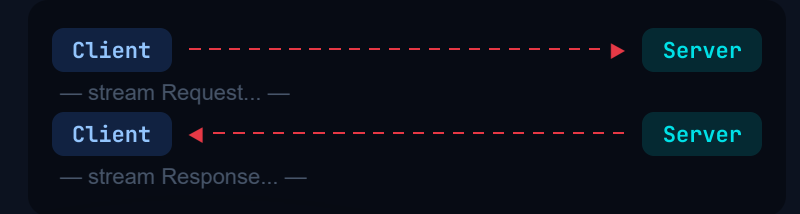
rpc `BulkCreate`(stream Req) returns(Res)

Projemizde: **Toplu sipariş** — gRPC-Web'de kısıtlı, kod tanımlı

Bidirectional

Çift Yönlü Akış

Eş zamanlı çoklu istek ↔ yanıt



rpc `LiveChat`(stream Req) returns(stream Res)

Projemizde: **Canlı sohbet** — gRPC-Web bidi desteği kısıtlı

Temel Kavramlar

Sistemi anlayabilmek için bilmeniz gereken 4 temel yapı taşı

- 1

Protocol Buffers (Protobuf)
Google'ın binary serileştirme formatı. JSON yerine binary — daha hızlı, daha küçük, tip güvenli.
- 2

Service Definition (.proto)
API sözleşmesi. Hangi metodlar var, ne alır, ne döner — dil bağımsız tek kaynak.
- 3

Stub & Server (Auto-generated)
protoc ile .proto'dan otomatik üretilen istemci ve sunucu kodları.
- 4

RPC Tipleri (4 mod)
Unary · Server Streaming · Client Streaming · Bidirectional Streaming

order.proto

```
// Servis tanımı
service OrderService {
  // Unary RPC
  rpc CreateOrder (CreateOrderRequest)
    returns (CreateOrderResponse);

  // Server Streaming RPC
  rpc TrackOrder (TrackOrderRequest)
    returns (stream OrderStatusUpdate);
}

// Mesaj tanımı
message CreateOrderRequest {
  string product_id = 1;
  int32 quantity = 2;
  string customer_id = 3;
}
```

SERVİSLER

OrderService

InventoryService

NotificationService

PORTLAR

:50051 Order

:50052 Inventory

:50053 Notification

Canlı Demo

Çalışan uygulamayı birlikte inceleyelim — localhost:8081



ORDER SERVICE

Sipariş Yönetimi

Sipariş oluştur → Stok kontrol et → Bildirim gönder. Unary + Server Streaming RPC kullanılıyor.

rpc CreateOrder → TrackOrder



INVENTORY SERVICE

Stok Yönetimi

Ürün ekle, listele, stok kontrol et, stok düşür. Gerçek zamanlı ürün grid görünümü.

rpc ListProducts → AddProduct



NOTIFICATION SERVICE (PYTHON)

Çapraz Dil Demo

Node.js'ten Python servisine gRPC çağrısı. Polyglot iletişim canlı olarak görülüyor.

Node.js → Envoy → Python

```
$ docker-compose up --build -d
Starting inventory-service ... done
Starting notification-service ... done
Starting order-service ... done
Starting envoy ... done
Starting frontend ... done
✓ Dashboard: http://localhost:8081
```

Zorluklar ve Çözümler

Geliştirme sürecinde karşılaştığımız gerçek engeller



Tarayıcı HTTP/2 Trailers Sorunu

Tarayıcılar native gRPC'yi desteklemez — HTTP/2 trailerlarını okuyamaz



Envoy Proxy Entegrasyonu

gRPC-Web isteklerini native gRPC'ye bridge eden Envoy yapılandırması



Çoklu Dil Proto Paylaşımı

Docker container'lar arası .proto dosyası erişimi — her dil kendi stubs'ına ihtiyaç duyar



Docker Bind Mounts

Multi-stage build + **Bind Mounts** ile tüm servisler aynı .proto kaynağını paylaşır



gRPC-Web Streaming Sınırları

Client-side streaming ve bidi streaming gRPC-Web'de tam destek görmez



Servis Tasarımı Revizyonu

Unary + Server Streaming önceliklendirilerek tam çalışan demo sağlandı



Sorularınızı Bekliyorum

Herhangi bir kavram, kod detayı veya mimari karar
hakkında soru sorabilirsiniz

[GitHub](#) emrebykdr/gRPC-Microservices

[Dashboard](#) localhost:8081

[Docs](#) grpc.io/docs